

Pruebas unitarias en java

M4: Fundamentos de programación en Java

|AE7: Implementar una suite de pruebas unitarias en lenguaje Java utilizando Junit para asegurar el buen funcionamiento de una pieza de software

Introducción

Estuvimos observando en unidades anteriores aspectos relacionados al diseño de nuestra aplicación, a su construcción y buenas prácticas, por lo que el contenido que aprenderemos a continuación estará orientado a asegurarnos que el código y sus funcionalidades sean operativas y detectar a tiempo potenciales fallas al modificar nuestro código.

En este sentido, para lograr lo mencionado existen los llamados test. Estos pueden ser funcionales o unitarios. Los primeros son aquellos que realizamos al momento de verificar la funcionalidad de nuestra aplicación y la capacidad de responder peticiones según como lo esperamos. Los segundos ayudan a medir que nuestro código esté cubierto por tests..

De esta manera la lección 7 tendrá como foco los test unitarios y explicar cómo se implementan con JUnit5.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Implementar una suite de pruebas unitarias en lenguaje Java utilizando JUnit5 para asegurar el buen funcionamiento de una pieza de software.

Características de las pruebas unitarias, ventajas y limitaciones

El Test Unitario es la prueba que nos va a permitir verificar una unidad o parte de nuestro código. Es decir, probar y verificar partes aisladas de nuestro código para asegurarnos que la funcionalidad que implementamos funciona correctamente.

Características

Las principales características a mencionar por parte de las pruebas unitarias, son:

- **Independencia:** Las pruebas unitarias deben ser independientes entre sí, lo que significa que cada prueba debe poder ejecutarse de forma aislada sin depender de otras pruebas o del entorno externo. Más adelante desarrollaremos ejemplos en cuanto a esto.
- **Automatización:** Las pruebas unitarias se diseñan para ser ejecutadas de forma automatizada. Se utilizan frameworks como JUnit, TestNG o Mockito para facilitar la creación y ejecución de las pruebas. En el caso de esta lección profundizaremos en las pruebas unitarias con JUnit5
- **Aislamiento:** Cada prueba unitaria debe probar una única funcionalidad o comportamiento específico. Para lograr esto, se utilizan mocks o stubs para aislar la unidad en prueba de sus dependencias externas.
- **Cobertura o Coverage:** Las pruebas unitarias deben cubrir la mayor cantidad de código posible. El objetivo es asegurar que todas las funcionalidades y caminos del código sean probados, minimizando la posibilidad de errores.
- **Repetitividad:** Las pruebas unitarias deben ser repetibles y producir resultados consistentes. Esto significa que deben poder ejecutarse en cualquier entorno sin depender de factores externos.
- **Rapidez:** Las pruebas unitarias deben ser rápidas de ejecutar, ya que se espera que se puedan ejecutar frecuentemente, incluso durante el proceso de desarrollo.

- **Legibilidad:** Es importante que las pruebas unitarias sean claras y comprensibles para facilitar su mantenimiento y comprensión por parte de otros desarrolladores.

Ventajas y Desventajas

Las principales **ventajas** a la hora de implementar Test unitarios en nuestro código, son:

1. **Detección temprana de errores:** Las pruebas unitarias te permiten identificar errores y problemas en el código de manera temprana. Al ejecutar las pruebas de forma regular, puedes capturar y corregir los errores en las primeras etapas del desarrollo, lo que facilita la solución de problemas y evita que se propaguen a otras partes del código.
2. **Mejora de la calidad del código:** Las pruebas unitarias fomentan el desarrollo de un código de alta calidad. Al escribir pruebas, debes pensar en la funcionalidad de forma aislada y en cómo probarla adecuadamente. Esto conduce a un código más modular, desacoplado y mantenible.
3. **Facilita el proceso de refactorización:** Las pruebas unitarias actúan como una red de seguridad al realizar refactorizaciones en el código. Siempre que realices cambios en tu código, puedes ejecutar las pruebas unitarias para asegurarte de que las modificaciones no hayan introducido nuevos errores.
4. **Documentación y comprensión del código:** Las pruebas unitarias bien escritas sirven como documentación viva de cómo se espera que funcione el código. Al leer las pruebas, otros desarrolladores pueden comprender rápidamente el comportamiento esperado de una determinada funcionalidad o clase.
5. **Facilita el trabajo en equipo:** Las pruebas unitarias permiten a los desarrolladores trabajar en paralelo y de manera colaborativa. Cada desarrollador puede escribir pruebas para las funcionalidades que está implementando y ejecutarlas para asegurarse de que no haya conflictos con otras partes del código.
6. **Mejora la productividad:** Aunque escribir pruebas unitarias requiere tiempo adicional, a largo plazo, puede mejorar la productividad del equipo. Al detectar y corregir errores tempranamente, se reducen los tiempos de depuración y se evitan problemas en etapas posteriores del desarrollo.

- 7. Facilita la integración continua y entrega continua:** Las pruebas unitarias son un componente clave de las prácticas de integración continua y entrega continua. Al tener un conjunto de pruebas sólidas, puedes automatizar su ejecución en los sistemas de integración continua, lo que garantiza que el código sea siempre estable y funcional.

Por otro lado las principales **desventajas** a la hora de implementar Test unitarios en nuestro código y las posibles soluciones son:

- 1. Costo de desarrollo:** La implementación de pruebas unitarias puede requerir un esfuerzo adicional de desarrollo. Es necesario invertir tiempo y recursos para escribir las pruebas y mantenerlas actualizadas a medida que el código evoluciona. Esto puede aumentar la carga de trabajo inicial y prolongar los plazos de entrega. Pero sabiendo que si son bien planteadas, otorgan la posibilidad de estabilidad de la app ante futuros cambios.
- 2. Curva de aprendizaje:** Para escribir pruebas unitarias efectivas, los desarrolladores deben familiarizarse con los frameworks y herramientas de pruebas, como JUnit o TestNG. Esto implica una curva de aprendizaje adicional para el equipo de desarrollo, especialmente si no están familiarizados con estas herramientas. Si bien es una desventaja, no es nada que no se pueda solucionar estudiando sobre Test Unitarios y practicando en nuestro día a día.
- 3. Mantenimiento continuo:** A medida que el código evoluciona, las pruebas unitarias también deben actualizarse y mantenerse. Si no se actualizan correctamente, las pruebas pueden volverse obsoletas y generar falsos positivos o falsos negativos. Esto requiere dedicar tiempo y esfuerzo para mantener el conjunto de pruebas actualizado y relevante.
- 4. Cobertura limitada:** Aunque las pruebas unitarias pueden cubrir una parte significativa del código, es posible que no logren cubrir todos los casos posibles. Algunos aspectos del sistema, como la interacción con bases de datos o servicios externos, pueden ser difíciles de probar mediante pruebas unitarias, lo que limita la cobertura total.
- 5. Sobrecarga de pruebas:** Si no se administra adecuadamente, el exceso de pruebas unitarias puede generar una sobrecarga en el proceso de desarrollo. Demasiadas pruebas pueden ralentizar el tiempo de compilación y ejecución, y puede ser difícil mantener todas las pruebas actualizadas y relevantes. Esto puede ser tranquilamente solucionado,

planteando bien los diseños y arquitecturas de la app, con el fin de evitar monolitos saturados en múltiples módulos y tests.

- 6. Falsos sentimientos de seguridad:** Aunque las pruebas unitarias son valiosas para detectar errores, no garantizan la ausencia total de errores en el código. Pueden pasar por alto ciertos escenarios o no detectar problemas de integración entre diferentes unidades. Es importante combinar las pruebas unitarias con otras formas de pruebas, como las pruebas de integración y las pruebas de aceptación. Esta, puede ser solucionada, con la correcta supervisión de los desarrolladores menos experimentados, con el fin de dar otra mirada crítica a la hora de ver, “Que se buscó cubrir y testear en las pruebas unitarias”

Podemos encontrarnos con distintas escuelas o líneas de pensamiento que expongan “Desventajas” de las pruebas unitarias, pero la realidad es que TODAS las desventajas, pueden ser reducidas, logrando destacar así las ventajas, que sobre todas las cosas, nos otorgarán integridad a nuestra App.

Introducción a JUnit

JUnit5 es un framework de pruebas unitarias para el lenguaje de programación Java. Es una herramienta ampliamente utilizada y reconocida en el desarrollo de software para automatizar y simplificar la creación, ejecución y evaluación de pruebas unitarias.

Las pruebas unitarias escritas con JUnit5 son programas Java que verifican el comportamiento y la funcionalidad individual de componentes aislados, como métodos, clases o incluso paquetes enteros de código. El objetivo de JUnit5 es garantizar que cada unidad de código funcione correctamente de forma independiente.

¿Qué conocer antes de escribir Test con JUnit?

- **Programación en Java:** Es fundamental tener un buen entendimiento de los conceptos básicos de programación en Java, como variables, tipos de datos, estructuras de control (if, bucles, etc.), clases, métodos y excepciones. JUnit se basa en el lenguaje Java, por lo que es necesario comprender y escribir código Java correctamente.
- **POO:** Dado que JUnit se utiliza para probar unidades de código a nivel de clase o método, es importante tener conocimientos de programación

orientada a objetos. Debes estar familiarizado con conceptos como clases, objetos, herencia, encapsulamiento y polimorfismo.

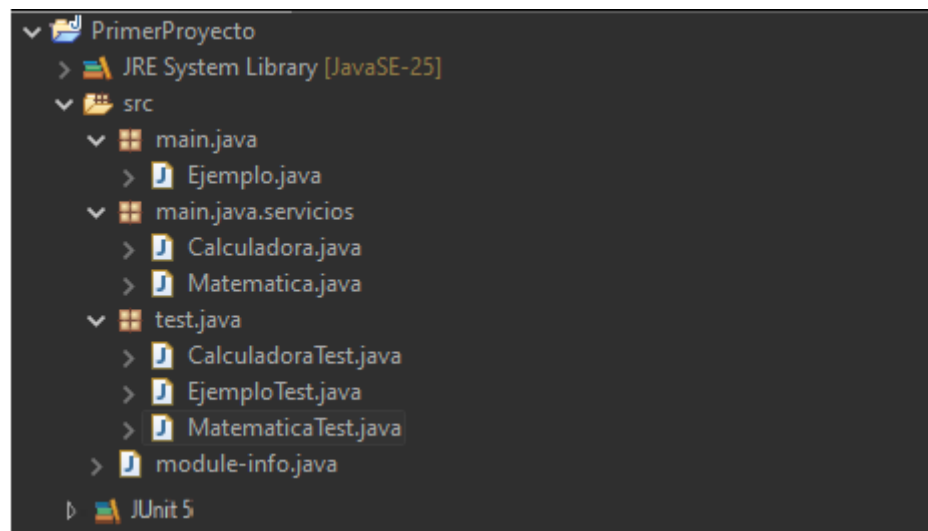
- **Método de prueba:** Antes de utilizar JUnit, es beneficioso entender qué es un método de prueba y cómo funciona. Un método de prueba es una función que verifica el comportamiento esperado de una unidad de código específica. Debes comprender cómo escribir métodos de prueba y cómo verificar resultados utilizando aserciones (assertions).
- **Anotaciones:** JUnit utiliza anotaciones Java para marcar los métodos de prueba y definir su comportamiento. Es importante comprender cómo funcionan las anotaciones en Java y cómo se aplican en el contexto de JUnit. Algunas anotaciones clave en JUnit son `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll` y `@AfterAll`, conceptos que más adelante explicaremos.
- **Aserciones (assertions):** Las aserciones son declaraciones que se utilizan para verificar si una condición es verdadera o falsa. JUnit5 proporciona un conjunto de métodos de aserción que se utilizan para evaluar y validar resultados en las pruebas unitarias. Es importante conocer los diferentes métodos de aserción disponibles en JUnit5 y cómo utilizarlos correctamente.

Creando un Caso de Prueba

A la hora de crear los casos de prueba para mi app, deberé tener en consideraciones algunos criterios previos, uno de ellos será que porcentaje de Coverage o cobertura estableceremos, un valor mínimo aceptable es más del 70%, entre el 85% y 95% es muy bueno y más del 95% es excelente.

Una vez definido este valor, deberemos considerar todas las clases a cubrir con test y que casos posibles vamos a contemplar en los test. A continuación se darán unos criterios de los casos de prueba y cómo organizar nuestros tests.

1. **Tantas clases Test como Clases a testear:** Dentro de nuestra App deberemos generar tantas ClasesTest como Clases dentro de nuestra App que deseamos testear. Un posible error al iniciarse en la creación de los unit test, es que dentro de una ClaseTest, busquemos testear tanto una Clase1 de nuestra app como otra Clase2, si deseamos testear ambas clases deberemos generar dos Clases Test distintas para cubrirlas.



2. **Funciones a testear:** Si la Clase a Testear en mi App tiene 5 funciones, mi ClaseTest deberá tener un mínimo de 5 @Test, NUNCA se debe integrar en un @Test más de una función a testear.

Por otro lado, algo que podría ser posible es dentro de un @Test, es que busquemos testear dos posibles salidas, un ejemplo de eso seria la siguiente función, parImpar, que puede devolver dos posibles valores, si analizamos el @Test veremos que posee dos assertEquals


```

MatematicaTest.java
18
19 @Test
20 void sumar() {
21     int expected = 4;
22     int actual = this.matematica.sumar(2, 2);
23     assertEquals(expected, actual);
24 }
25
26 @Test
27 void restar() {
28     int expected = 4;
29     int actual = this.matematica.restar(10, 6);
30     assertEquals(expected, actual);
31 }
32
33 @Test
34 void dividir() {
35     double expected = 2;
36     double actual = this.matematica.dividir(8, 4);
37     assertEquals(expected, actual, 0.001);
38 }
39
40 @Test
41 void multiplicar() {
42     int expected = 6;
43     int actual = this.matematica.multiplicar(3, 2);
44     assertEquals(expected, actual);
45 }
46
47 @Test
48 void parImpar() {
49     String expected = "Par";
50     String actual = this.matematica.parImpar(2);
51     assertEquals(expected, actual);
52
53     expected = "Impar";
54     actual = this.matematica.parImpar(1);
55     assertEquals(expected, actual);
56 }
57 }
58

Matematica.java
1 package main.java.servicios;
2
3 public class Matematica {
4
5     public int sumar(int a, int b) {
6         return a+b;
7     }
8
9     public int restar(int a, int b) {
10        return a-b;
11    }
12
13    public double dividir(int dividendo, int divisor) {
14        return dividendo/divisor;
15    }
16
17    public int multiplicar(int a, int b) {
18        return a*b;
19    }
20
21    public String parImpar(int num) {
22        if (num % 2 == 0) {
23            return "Par";
24        } else {
25            return "Impar";
26        }
27    }
28
29 }
30

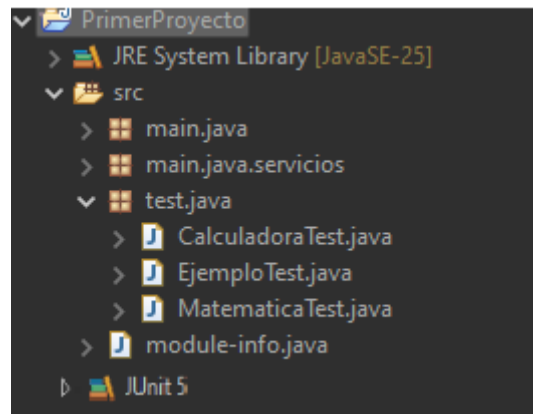
```

3. **Casos de prueba:** Una vez definida la clase donde buscaré cubrir los test, se deberá entender que funciones quiero cubrir, qué parámetros de entrada necesitan las mismas, que parámetros de salida tendrán y qué posibles errores pueden ocurrir al usarlas.

Integración de JUnit en Eclipse

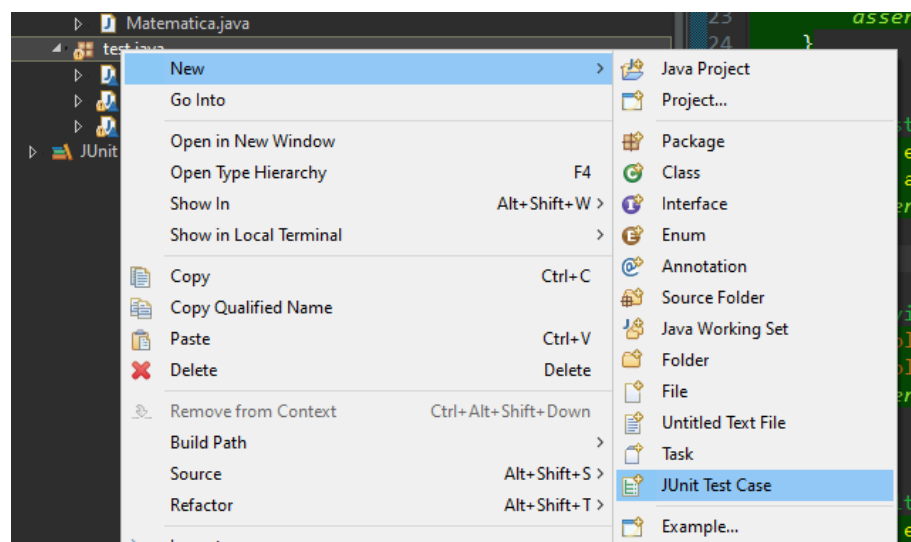
Una de las principales ventajas de desarrollar java en Eclipse, es que es un IDE con múltiples integraciones, una de ellas es JUnit, en las últimas versiones de Eclipse y en la mayoría de los casos, no es necesario ningún tipo de integración adicional. Solo deberemos tener las siguientes consideraciones:

- **Versión de JUnit:** Es importante controlar qué versión de JUnit estamos utilizando, es recomendable siempre tratar de utilizar JUnit 5, pero si se usa una versión anterior, prestar atención a las @Anotaciones a usar.
- **Gestión de Paquetes:** Este punto busca más que nada que nuestra App esté ordenada, eso lo podremos lograr si se logra separa los test del resto de la app, como en el siguiente ejemplo

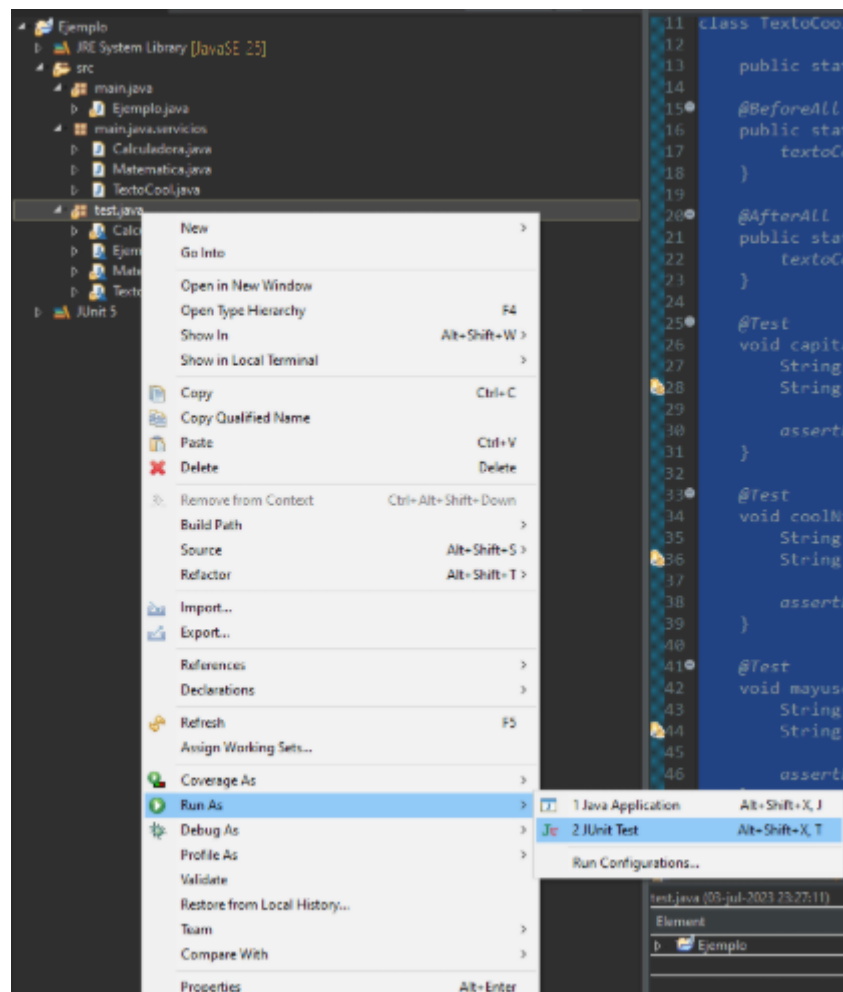


La App se llama “Ejemplo” y dentro de la App tenemos dos paquetes principales dentro de “src”, main y test. Dentro de main tendremos nuestra App en sí, con su organización de paquetes, como es el paquete servicios dentro de main/java. Y por otro lado el paquete test tendrá un sub paquete llamado java, que es donde se contendrán los clases Test que buscarán testear las clases de nuestra App

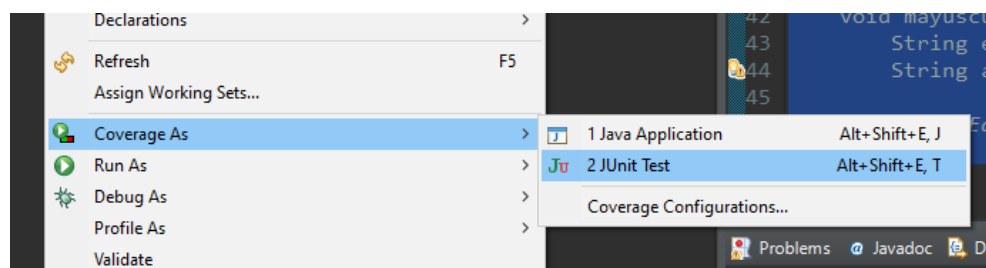
- **Generate Test Cases:** La manera más sencilla de generar un test case o ClaseTest, es haciendo click derecho sobre el paquete donde quiero generarlo y Colocarle el nombre correspondiente [NombreOriginalDeLaClase]+Test.



- **Corriendo los Test::** En este ítem queremos compartirles como correr todos los test de forma sencilla desde Eclipse. En caso de querer correr los test de una sola clase, realizar click derecho desde la clase deseada.



- **Corriendo los Test:** Si se desea ver el coverage de los test realizados, se deberá correr “Coverage As”. Y se verá lo siguiente:



test.java (03-jul-2023 23:43:19)				
Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▲ Ejemplo	90,6 %	329	34	363
▲ src	90,6 %	329	34	363
▲ main.java.servicios	83,8 %	155	30	185
▸ Calculadora.java	0,0 %	0	30	30
▸ Matematica.java	100,0 %	36	0	36
▸ TextoCool.java	100,0 %	119	0	119
▲ main.java	0,0 %	0	4	4
▸ Ejemplo.java	0,0 %	0	4	4
▲ test.java	100,0 %	174	0	174
▸ CalculadoraTest.java	100,0 %	4	0	4
▸ EjemploTest.java	100,0 %	25	0	25
▸ MatematicaTest.java	100,0 %	104	0	104
▸ TextoCoolTest.java	100,0 %	41	0	41

Utilización de Fixtures en las unidades de prueba

En el contexto de las pruebas unitarias y específicamente en JUnit, los "fixtures" son objetos o estructuras de datos que se utilizan para establecer un estado inicial predefinido antes de ejecutar las pruebas y restaurar el estado original después de que se completen las pruebas. Los fixtures son esenciales para garantizar que las pruebas se ejecuten de manera aislada y consistente.

En JUnit, los fixtures se definen utilizando anotaciones y métodos especiales que se ejecutan antes y después de cada prueba o de la suite de pruebas.

@Anotaciones en JUnit

Dentro de JUnit5 encontraremos diversas anotaciones, las cuales pueden ser reconocidas ya que empiezan con "@". Estas anotaciones son parte del framework y nos facilitan muchas cosas a la hora de escribir un test, JUnit tiene muchas anotaciones, pero será importante prestar atención más que nada a las siguientes.

- **@Test:** Esta anotación se utiliza para marcar un método como una prueba unitaria. Los métodos anotados con **@Test** deben ser públicos y no devolver ningún valor.
- **@BeforeEach:** Esta anotación se utiliza para marcar un método que se ejecuta antes de cada prueba. Puedes usarlo para realizar configuraciones o inicializaciones necesarias antes de cada prueba.
- **@AfterEach:** Esta anotación se utiliza para marcar un método que se ejecuta después de cada prueba. Puedes usarlo para limpiar recursos o realizar acciones posteriores a cada prueba.

```

1 package test.java;
2
3 import static org.junit.Assert.assertEquals;
4 import static org.junit.Assert.assertNotEquals;
5
6 import org.junit.jupiter.api.AfterEach;
7 import org.junit.jupiter.api.BeforeEach;
8 import org.junit.jupiter.api.Test;
9
10 class EjemploTest {
11
12     // private MiClase miObjeto;
13
14     @BeforeEach
15     public void configuracionInicial() {
16         // Metodo que se correra antes de cada @Test
17         // miObjeto = new MiClase();
18     }
19
20     @AfterEach
21     public void limpiezaFinal() {
22         // Accion final que se ejecutara al final de cada @Test
23         // miObjeto = null; Liberar el objeto
24     }
25
26     @Test
27     void test1() {
28         int expected = 1;
29         int actual = 1;
30         assertEquals(expected, actual);
31     }
32
33     @Test
34     void test2() {
35         int unexpected = 2;
36         int actual = 1;
37         assertNotEquals(unexpected, actual);
38     }
39 }
40
41

```

- @BeforeAll: Esta anotación se utiliza para marcar un método que se ejecuta una sola vez antes de todas las pruebas en una clase. El método anotado debe ser estático.
- @AfterAll: Esta anotación se utiliza para marcar un método que se ejecuta una sola vez después de todas las pruebas en una clase. El método anotado debe ser estático.

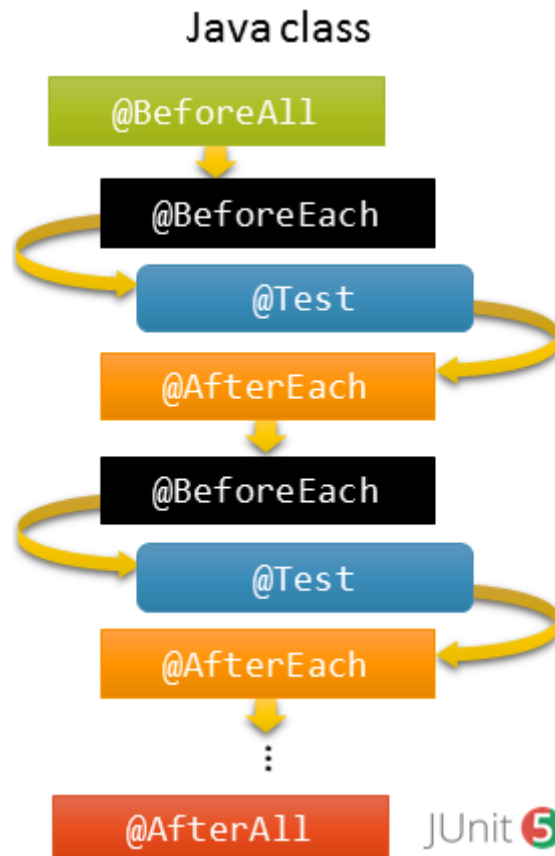
```

1 package test.java;
2
3 import static org.junit.Assert.assertEquals;
4 import static org.junit.Assert.assertNotEquals;
5
6 import org.junit.jupiter.api.AfterAll;
7 import org.junit.jupiter.api.BeforeAll;
8 import org.junit.jupiter.api.Test;
9
10 class EjemploTest {
11
12 // private MiClase miObjeto;
13
14 @BeforeAll
15 public void configuracionInicial() {
16 // Metodo que se corra una unica vez antes de que se corran los @Test
17 // miObjeto = new MiClase();
18 }
19
20 @AfterAll
21 public void limpiezaFinal() {
22 // Accion final que se corra cuando se corra el ultimo @Test
23 // miObjeto = null; liberar el objeto
24 }
25
26 @Test
27 void test1() {
28     int expected = 1;
29     int actual = 1;
30     assertEquals(expected, actual);
31 }
32
33 @Test
34 void test2() {
35     int unexpected = 2;
36     int actual = 1;
37     assertEquals(unexpected, actual);
38 }
39 }
40
41

```

- **@DisplayName:** Esta anotación se utiliza para proporcionar un nombre descriptivo para una prueba o una clase de prueba. Puedes usarlo para hacer que el nombre de la prueba sea más legible y significativo.
- **@Disabled:** Esta anotación se utiliza para deshabilitar una prueba o una clase de prueba. Puedes usarlo cuando desees omitir temporalmente una prueba sin eliminarla por completo.
- **@ParameterizedTest:** Esta anotación se utiliza para marcar un método como una prueba parametrizada. Permite ejecutar la misma prueba con diferentes conjuntos de parámetros.
- **@RepeatedTest:** Esta anotación se utiliza para marcar un método como una prueba repetida. Puedes especificar el número de repeticiones de la prueba.
- **@Timeout:** Esta anotación se utiliza para establecer un límite de tiempo para una prueba. Si la prueba excede el tiempo especificado, se considera fallida.

A modo de representación gráfica, este es el orden de ejecución ante cuando se corren los tests:



Utilización de objetos simulados (mocks)

Un Mock es un objeto preprogramado con expectativas que conforman la especificación acerca de cómo se espera que se reciban las llamadas. Se utilizan mocks en los test para poder indicar que necesitamos un objeto simulado para las pruebas. Dentro de Spring Boot existe Mockito que nos ayuda a simular pruebas teniendo en cuenta, por ejemplo, qué tipos de datos enviamos y qué tipos de datos esperamos como respuesta.

En síntesis, es un tipo concreto de doble de test.

¿Qué conceptos podemos ver dentro de un test unitario con mocks?

Existen otros conceptos de test aparte de Mock, que son doble de Test. Existen diferentes especificaciones que se detallan a continuación:

Dummy: se considera como argumento pero nunca se usa realmente. Normalmente, los objetos dummy se usan sólo para rellenar listas de parámetros.

Fake: tiene una implementación que realmente funciona pero, por lo general, toma algún atajo o cortocircuito que le hace inapropiado para producción (como una base de datos en memoria, por ejemplo).

Stub: proporciona respuestas predefinidas a llamadas hechas durante los tests, frecuentemente, sin responder en absoluto a cualquier otra cosa fuera de aquello para lo que ha sido programado. Los stubs pueden también grabar información sobre las llamadas; tal como una pasarela de email que recuerda cuántos mensajes envió.

Hay que salvar diferencias entre Stub y Mock aunque parezcan similares.

Podría decirse que el stub es como un mock con menor potencia, un subconjunto de su funcionalidad. Mientras que en el mock podemos definir expectativas con todo lujo de detalles, el stub tan solo devuelve respuestas preprogramadas a posibles llamadas. Un mock valida comportamiento en la colaboración, mientras que el stub simplemente simula respuestas a consultas.

A continuación, veremos un ejemplo práctico de cómo mockear:


```

class CalculadoraTest {

    @Mock
    private Matematica matematica;

    public CalculadoraTest() {
        MockitoAnnotations.openMocks(this);
    }

    @Test
    public void sumarRestar() {

        // Configuración del mock
        when(matematica.sumar(0,0)).thenReturn(100);
        when(matematica.restar(0,0)).thenReturn(4);

        // Creación del objeto bajo prueba
        Calculadora calculadora = new Calculadora(matematica);

        // Ejecución del método bajo prueba
        int actual = calculadora.sumarRestar("Sumar", 50, 50);
        assertEquals(10, actual);

        actual = calculadora.sumarRestar("Restar", 12, 8);
        assertEquals(0, actual);
    }
}

```

Suites de pruebas

Anteriormente vimos lo que eran Casos de prueba o Test Cases, ahora le toca el turno a Suite de Prueba (Test Suite).

¿Qué es una Suite de pruebas?

Una Test Suite es una colección o **agrupación de casos de prueba relacionados que se ejecutan juntos como una unidad**. En lugar de ejecutar cada caso de prueba de forma individual, una Test Suite permite ejecutarlos en conjunto, lo que facilita la organización y automatización de las pruebas.

¿Para qué sirve una Test Suite?

Una Test Suite sirve para varios propósitos importantes en las pruebas de software:

Organización: Permite organizar y estructurar las pruebas de manera lógica y jerárquica, facilitando la navegación y comprensión de las pruebas en un proyecto.

Reutilización: Permite reutilizar casos de prueba en diferentes contextos y combinaciones. Puedes combinar y reutilizar suites de prueba existentes para crear suites más grandes y abarcar diferentes escenarios.

Ejecución eficiente: Al agrupar casos de prueba relacionados en una suite, puedes ejecutar un conjunto completo de pruebas de una vez. Esto ahorra tiempo y agiliza el proceso de ejecución.

Informes y resultados agregados: Las suites de prueba proporcionan informes y resultados agregados de los casos de prueba individuales. Esto facilita la identificación de casos fallidos y problemas en general.

¿Cómo se plantean las Test Suites?

La forma en que se plantean las Test Suites puede variar según la herramienta de prueba utilizada. Aquí hay una guía general sobre cómo plantear una Test Suite:

Identificar casos de prueba relacionados: Agrupa los casos de prueba que están relacionados y se centran en probar una funcionalidad o un componente específico.

Crear una clase de suite: Crea una nueva clase de suite que sirva como punto de entrada para la ejecución de la suite. Esta clase debe estar anotada adecuadamente según la herramienta de prueba que estés utilizando (por ejemplo `@ExtendWith` en JUnit 5).

Especificar los casos de prueba incluidos: En la clase de suite, especifica los casos de prueba individuales que deseas incluir en la suite. Puedes hacer esto utilizando anotaciones o métodos específicos proporcionados por la herramienta de prueba.

Configurar y ejecutar la suite: Configurar cualquier configuración adicional necesaria para la suite y ejecutala. Esto puede implicar la configuración de parámetros de ejecución, establecimiento de datos iniciales, etc.

Una Test Suite es una colección de casos de prueba relacionados que se ejecutan juntos para probar una funcionalidad o un componente del software. Las suites de prueba son útiles para organizar, reutilizar y ejecutar eficientemente casos de

prueba, y proporcionan informes agregados para una visión general de los resultados de las pruebas.

El Desarrollo Dirigido por Test (TDD)

TDD o Test-Driven Development (Desarrollo Dirigido por Tests) es una práctica de programación que consiste en escribir primero las pruebas (generalmente unitarias), después escribir el código fuente que pase la prueba satisfactoriamente y, por último, refactorizar el código escrito.

Básicamente, a este ciclo también se lo conoce como rojo (hacer que la prueba falle), verde (hacer que la prueba pase) y refactor del código.

Un concepto opuesto, pero generalmente bastante usado, es BDD (Behavior Driven Development), donde se desarrolla primero la implementación y luego los tests.

¿Cómo se crea un ejemplo de TDD en un proyecto real?

Para ejemplificar, se debe pensar en una serie de pasos:

Supongamos que el cliente nos pide que desarrollemos una calculadora que sume números.

Acordamos con el cliente que el criterio de aceptación sería que si se introduce en la calculadora dos números y se le da la operación de suma, la calculadora muestre el resultado de la suma en la pantalla.

Teniendo en cuenta este criterio, se comienza a definir el funcionamiento del algoritmo de suma y convertimos el criterio de aceptación en una prueba concreta, por ejemplo, un algoritmo que si se introduce un 3 y un 5 te devuelva un 8:

Así es como definimos el código de criterio de aceptación dentro de un test:

```
public void testSuma() { assertEquals(8,
    Calculadora.suma(3,5)); }
```

Luego ya es más sencillo poder definir el código productivo, que sería el siguiente:

```
public class Calculadora { public static  
    int b) { return a+b; } }
```

Luego de haber hecho estos pasos, verificamos si nuestro test lo logró. Al ejecutar, nos debería dar el flag verde.

Posteriormente, podemos aplicar un refactor al código en el test. Veamos un ejemplo aplicando el 3 A's:

```
public void testSuma() {  
  
    //Arrange  
    int a = 3;  
    int b = 5;  
  
    //Act  
    int resultado = Calculadora.suma(a,b);  
  
    //Assert  
    assertEquals(8, resultado);  
}
```

Cierre

A modo de síntesis, pudimos aprender la importancia de la presencia del Unit Testing dentro de nuestras aplicaciones, si bien pueden llegar a mencionarnos desventajas respecto a la implementación, ahora sabemos que son más los beneficios que los problemas que trae.

Por otro lado pudimos conocer más funcionalidades de cómo funciona JUnit5 y sus anotaciones, sabemos identificar cuándo y cómo usarlas para lograr que nuestra ClaseTest, logre testear lo que deseamos.

Y por último, conocimos cómo solucionar problemas de dependencias mediante el uso de Mocks y conocimos el concepto de TDD, filosofía de desarrollo que nos permite poder anteponerse a problemas de nuestras funciones mediante el testing.

¡Nos vemos de nuevo, más adelante! 🙌

Referencias

- Baeldung. (s. f.). A guide to JUnit 5. Recuperado el 25 de julio de 2025, de <https://www.baeldung.com/junit-5>
- JUnit Team. (s. f.). JUnit [Sitio web]. Recuperado el 25 de julio de 2025, de <https://junit.org/>

¡Muchas gracias!

Nos vemos en la próxima lección

